



Karnataka State Police
Government of Karnataka

Powered by **H2S**

0atathon 2026

Technology partner



Team Details

Team name: <TEAM NAME>

Team leader name: <LEADER NAME>

Team size: <N>

Problem Statement: Challenge 01 — Intelligent Conversational AI for the KSP Crime Database

Veritas · a conversational crime-intelligence console where every claim traces to a record



Brief about the solution

An officer asks a question in English or Kannada. Veritas answers from the KSP FIR database, and every claim in the answer carries a citation to the record it came from.

The problem the schema creates

- The organizers' ER has no person. An Accused row belongs to exactly one case and its PersonID is a per-case sort label, so on the raw schema every offender is a first-timer, nobody co-offends with anybody, and “does he have priors?” has no answer.
- So identity is inferred, not looked up: Fellegi–Sunter probabilistic record linkage reconstructs people from Accused rows at F1 0.989. It runs as a batch pass right after load, and priors, the co-offender network, the money trail and every risk feature stand on it.

What that buys the investigator

- Ask in plain language, get the answer plus the map, network, money flow or forecast that supports it — and the reasoning trace that produced it.
- When the records do not support an answer, it says so. It does not return the nearest-looking case.
- Live on Catalyst: 37 tables, 10,000 FIRs, 105k rows, 6 officer ranks, 200 tests green.



Opportunities

How it differs from existing ideas

- It is not text-to-SQL over the FIR tables. Retrieval is HippoRAG (Gutiérrez et al., NeurIPS 2024): the query's entities seed Personalized PageRank over a knowledge graph built from the records, so one graph pass does multi-hop retrieval without an LLM in the loop. Deep questions escalate to Think-on-Graph (ICLR 2024) beam search, which returns a traceable reasoning path rather than only an answer.
- There is deliberately no model-authored query: ZCQL has no bind parameters, so a generated query would be the one place user text reaches the evidence store uninterpolated.

How it solves the problem

- 27 of the organizers' tables load verbatim; 10 vx_ tables add what the ER cannot express — the resolved person, the co-offending graph, the financial layer, session memory and the audit trail. Questions about a person, a network or a money flow become answerable at all.

Unique selling points

- Refusal is a designed path. A CRAG-style evaluator accepts, widens, or states “not found in the available records”; a named FIR that does not exist is refused, never approximated.
- Access control is enforced at query-construction time, not by masking the output — you cannot un-traverse a graph or reliably redact a name out of generated prose.
- Kannada speech and translation run inside our own container; no FIR text and no map coordinate ever reaches a third-party service.
- The audit log is a SHA-256 hash chain, re-verified by a Catalyst Cron job every 12 hours.



Features

- Conversational search — English and Kannada, streamed token by token, with numbered citation chips that open the source record in an evidence drawer.
- Voice — push-to-talk Kannada ASR (faster-whisper) and NLLB-200 translation, both in-container; the LLM never sees Kannada.
- Identity — “Ramesh Gowda” and “Ramesha Gouda” resolve to one person: priors, aliases, full case history, F1 0.989.
- Criminal network — PageRank, betweenness and Louvain communities over the co-offending projection; groups are labelled “Community 47”, because the ER records no gang and we do not invent one.
- Hotspots — KDE plus DBSCAN (eps 500 m) rendered on a self-drawn basemap, so FIR coordinates never travel inside a tile-service URL.
- Forecasting — Prophet with MinT reconciliation: a district's forecast always equals the coherent sum of its stations' forecasts.
- Risk — XGBoost with TreeSHAP explanations, a calibrated LightGBM 180-day recidivism model, and Isolation Forest district spike alerts pushed over WebSocket.
- Financial crime — an explainable structuring detector a court can audit line by line, alongside a GNN that catches coordinated multi-account layering the rule structurally cannot see; Sankey money flow.
- Investigation Copilot — case timeline, top-5 similar past cases with outcomes, ranked leads (direct co-accused only, so a lead is actionable this week), and a paste-ready case-diary paragraph.
- Governance — six officer ranks, tamper-evident audit chain, Aequitas bias audit, PDF case brief via SmartBrowz.



Process flow



Answer path. The evaluator has three outcomes: accept → synthesise a cited answer; widen and retry; or refuse — “not found in the available records”. A missed exact FIR lookup always refuses.

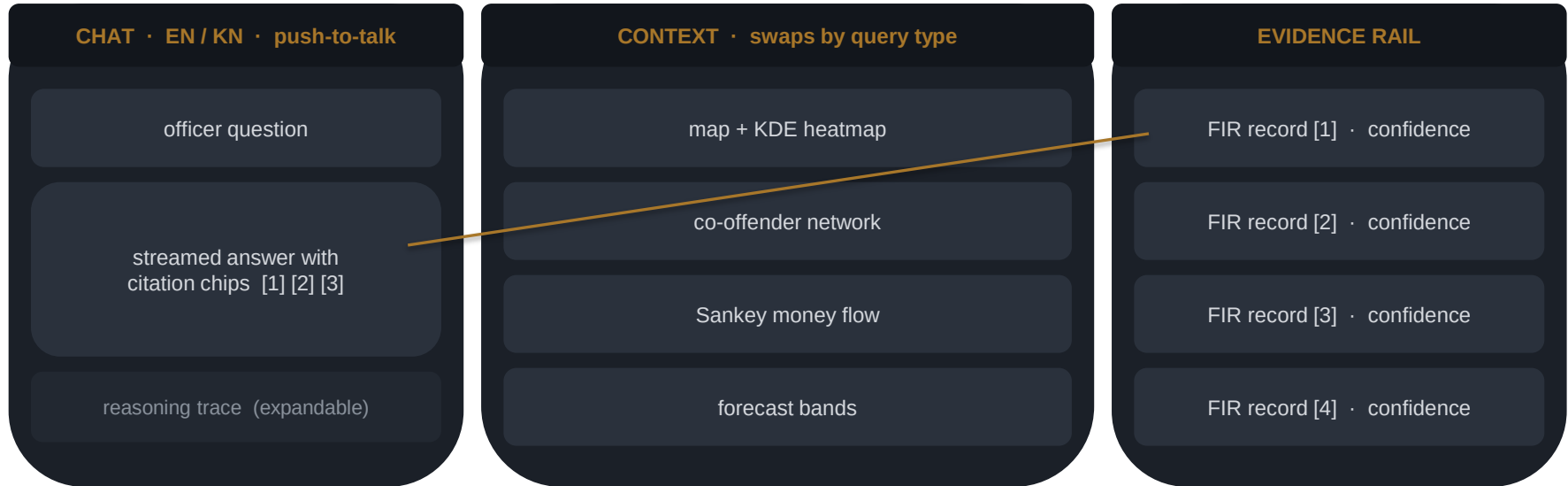


Build path, and the order is the architecture: each step depends on the last, and identity resolution cannot move — the graph, the money layer and every risk feature are built from resolved people, not from Accused rows.

Use case — a Sub-Inspector opens an unsolved chain-snatching FIR, presses Ask about this case, and gets: the timeline, five closed cases with the same modus operandi and their outcomes, the accused's resolved priors under two spellings of his name, his direct co-accused, and the paragraph to paste into the case diary. Everything is cited; a DSP asking the same question sees all 76 stations, the Investigating Officer sees one.



Console wireframe



Signature element — the evidence thread: selecting a citation draws a line from the claim to the record it rests on. Ink ground with a brass accent taken from the KSP uniform; fonts are resolved into the static export, so the page makes no third-party request. Live screenshots on slide 11.



Architecture

KSP OFFICERS (browser, 6 ranks)

COMMAND CONSOLE · Next.js static export on Catalyst Web Client Hosting · chat / map + graph + Sankey / evidence rail

FASTAPI on Catalyst AppSail (custom OCI image) · Catalyst Authentication · policy enforcement · SSE + WebSocket · audit hash chain

LANGGRAPH MULTI-AGENT ENGINE · Orchestrator → HippoRAG → Think-on-Graph (if low confidence) → specialist agents → CRAG evaluator → synthesis

DATA STORE
37 tables, 105k rows
ZCQL

FILE STORE
model weights
streamed at cold start

CACHE
session focus
read every turn

QUICKML
GLM-4.7-Flash
no API key in image

CRON
refresh 6h
audit verify 12h

IN-PROCESS, ON CATALYST COMPUTE · NetworkX graph · numpy vector search · Fellegi–Sunter · XGBoost / LightGBM / Prophet / DoWhy · faster-whisper + NLLB

Libraries, not third-party services: every model runs inside our own AppSail container. Reads run against a local sqlite mirror hydrated from the Data Store once per container, because live ZCQL refuses JOINS between value-related tables; the Data Store stays the record of truth.



Technologies

- Console — Next.js (output: export), TypeScript, MapLibre GL on a self-drawn basemap, ECharts, D3 force layout, IBM Plex resolved at build time.
- API — Python 3.11, FastAPI, SSE + WebSocket, shipped as a custom OCI image (0.88 GB).
- Reasoning — LangGraph; HippoRAG (NeurIPS 2024); Think-on-Graph (ICLR 2024); a Corrective-RAG evidence evaluator; GLM-4.7-Flash served by QuickML.
- Graph and retrieval — NetworkX (PageRank, Louvain, pivot-sampled betweenness, Personalized PageRank) over an edge table; numpy exact cosine with an IDF-weighted lexical half, so “IPC 457” still matches what an investigator actually types.
- Analytics — XGBoost + TreeSHAP, LightGBM, Prophet + MinT reconciliation, scikit-learn (KDE, DBSCAN, Isolation Forest), DoWhy, a PyTorch GNN for AML, Aequitas for the bias audit.
- Kannada — faster-whisper via CTranslate2 and NLLB-200 int8; ~760 MB of weights live in Catalyst File Store and are streamed and spliced in at cold start, never baked into the image.
- Data — Catalyst Data Store over ZCQL, with a schema-typed sqlite mirror for reads; synthetic FIRs generated on real ground truth (NCRB Karnataka IPC distributions, Census 2011 district socioeconomics, KA-GIS boundaries).
- Engineering — pytest, 200 tests green with no database and no Docker: the same ZCQL strings the deployed service sends run against sqlite, so the access-control rules are checked on every commit. Deploys relay through GitHub Actions.



Catalyst services used

- AppSail (custom OCI runtime) — the FastAPI service; 0.88 GB image, 2048 MB memory.
- Web Client Hosting — the console, served from /app/.
- Catalyst Authentication — says who signed in; the ER's Employee row stays authoritative for role and station. A Catalyst account with no Employee row is not an officer.
- Data Store — 37 tables, 105k rows, all reads and writes in ZCQL.
- File Store — the ASR and translation weights, in 8 chunks, streamed at cold start.
- Cache — per-session focus, read on every turn before the orchestrator routes anything.
- QuickML LLM Serving — GLM-4.7-Flash. The SDK authenticates as the app, so there is no API key in the image.
- Cron — veritas_refresh (6 h) and veritas_audit_verify (12 h), because a tamper check nobody runs is not a tamper check.
- SmartBrowz — PDF case-brief export. Admin API — provisions all 37 tables from one command.

Four capabilities have no Catalyst equivalent and therefore run inside our own container: Kannada ASR / translation (Zia has no speech or translation service), the vector index (QuickML's RAG is a managed upload pipeline with no custom retrieval hook), the knowledge graph (no Catalyst service is a graph database), and audit immutability (Data Store has no rules or triggers, so it was rebuilt as a hash chain in the data). Each is an absence, not a preference.



Estimated implementation cost

Prototype, as deployed — ₹0 recurring.

- AppSail (single instance, 2048 MB), Data Store, File Store, Cache, Cron and Web Client Hosting all sit inside the Catalyst development tier.
- No third-party subscriptions to pay for: no Gemini key, no map-tile service, no speech-to-text vendor, no managed graph or vector database.
- QuickML inference is the only per-call cost, and it is optional by construction — every provider failure degrades to the deterministic intent templates plus extractive synthesis, which still produce grounded, cited answers. The LLM makes an answer fluent; it never makes it true.

Production, per district cluster — what changes at CCTNS volume.

- Compute: AppSail instances scale with concurrent officers; the ML and Kannada stack adds no per-call service cost because it runs in the same container as the API.
- Storage: Data Store rows scale linearly with FIR volume; the entity-resolution and graph passes are batch jobs on Cron, so their cost is scheduled, not per-query.
- Human cost is the real line item and it is deliberate: every output is decision-support requiring an officer's judgement, so the system adds review time rather than removing it.



Prototype snapshots

Live console · replace each frame with its screenshot (?as=DSP on the deployed link signs in without credentials).

CITED ANSWER

citation chips + the evidence thread

CO-OFFENDER NETWORK

resolved people, Louvain communities, PageRank sizing

HOTSPOT MAP

KDE heatmap + case points on the self-drawn basemap

REFUSAL

a FIR number that does not exist

KANNADA

ಕನ್ನಡ question, English evidence, Kannada answer

RBAC CONTRAST

the same question as IO (1 station) and as DSP (76)



Performance and benchmarking

- Entity resolution, against the generator's answer key — F1 0.989 (precision 0.997, recall 0.981). This is the load-bearing number: without it there are no priors, no network and no money trail.
- Knowledge graph — 16,918 nodes and 87,120 edges, materialised in under a second. Louvain finds 12 communities of realistic size. (An earlier generator drew co-accused independently, making the network a random graph; Louvain correctly put 254 of 255 people in one community — the model was right and the data was wrong.)
- Retrieval — 13,835 documents, exact brute-force cosine, so there is no approximate-index recall loss to explain away at this scale.
- Exact lookup — FIR 100222201202600022 returns the Mandya Hurt case (PS 2201, filed 30 Jun 2026) at confidence 0.97; a well-formed FIR number that does not exist is refused rather than answered with the nearest narrative.
- Access control, measured live — an Investigating Officer sees 81 cases across 1 station; a DSP and above sees 500 across 76.
- Kannada — ಕನ್ನಡ query round-trips to English and back, routed as CRIME_SEARCH with 5 citations.
- Tests — 200 green, no database and no Docker required. Deployed image 0.88 GB, which fits AppSail's measured ~1.3 GB bundle sandbox.
- Live health — llm=quickml(glm-4.7-flash) · datastore=catalyst · firs=10000 · graph 16,918n/87,120e · vectors 13,835 docs · cache=catalyst.



Links

GitHub (public)

- <https://github.com/baveshraam/Veritas>

Deployed on Catalyst

- Console — <https://veritas-60077763394.development.catalystserverless.in/app/index.html>
- API — <https://veritas-api-50043864344.development.catalystappsail.in/health>

Demo video (3 minutes)

- <PASTE UNLISTED YOUTUBE OR PUBLIC DRIVE LINK>

Try it without credentials: append ?as=DSP to the console URL to sign in at that rank, or ?as=IO to see the same questions answered under a single station's access.



Additional details and future development

Next, in order of value

- Live CCTNS ingestion — Kafka topics per event type with Flink enrichment, turning the batch identity-resolution pass into a streaming one so a new FIR joins the network as it is filed.
- Iceberg or MinIO once volume and time-travel needs exceed the Data Store; Keycloak for HR-federated identity; OPA/Rego when the policy set outgrows a Python module; MLflow when the model count outgrows ad-hoc endpoints.
- Spatio-temporal GNN forecasting beyond Prophet + MinT, once historical volume justifies the training cost.

What we would insist on before real deployment

- The numbers here are measured on synthetic FIRs built on real ground truth, because no real FIR data was available to us. On live records the entity-resolution thresholds must be re-tuned against a labelled sample first: a false merge is two people's histories fused into one, and that is a wrongful-suspicion error, not a metric.
- The bias audit runs on geographic subgroups as well as demographic ones, because geography is where the over-policing feedback loop hides. That result should be published alongside any deployment.



Responsible AI

Predictive policing has a documented history of laundering historical bias — over-policing produces more recorded crime, which “predicts” crime in the same place — into an apparently objective score. The rules below are enforced in code and covered by tests, not asserted in a policy document.

- No protected or proxy attribute reaches a model. CasteID and ReligionID are stored because the ER declares them and conformance is a hard requirement; no model reads them. Storing is not scoring. Gender is kept only as a subgroup label, so the fairness audit has an axis to test against.
- Detector output is never the generator's input. The injected laundering patterns are written to a file, never to the column the detectors read — a classifier trained on a label sitting in the column it scores measures nothing.
- Aequitas (Saleiro et al. 2018, the toolkit built for criminal-justice risk tools) audits disparate impact and FPR/FNR parity across demographic and geographic subgroups.
- Human in the loop by design: every prediction is decision-support with confidence intervals and SHAP explanations, nothing is an automated trigger, and the console visibly distinguishes “the model suggests” from “the record shows”.
- The causal layer names its unmeasured confounder — police strength is not published per district in India — rather than adjusting for a column we would have had to fabricate.



Karnataka State Police
Government of Karnataka

Powered by **h2s**

Datathon

2026

Technology partner



THANK YOU